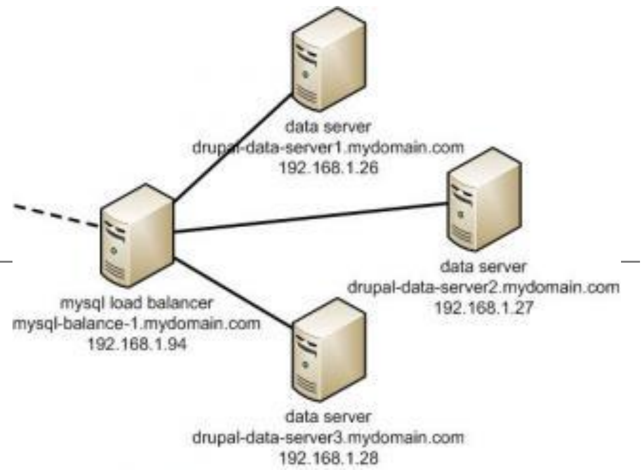
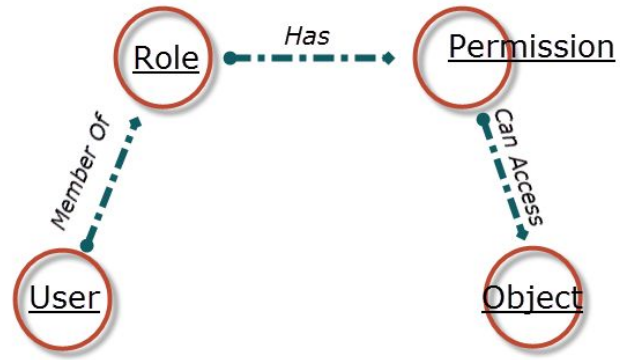
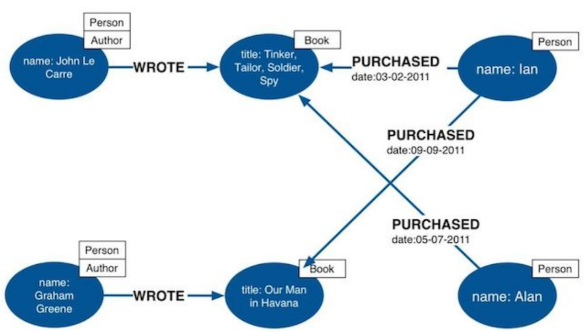
Ch 8: Graph Database : Neo4j

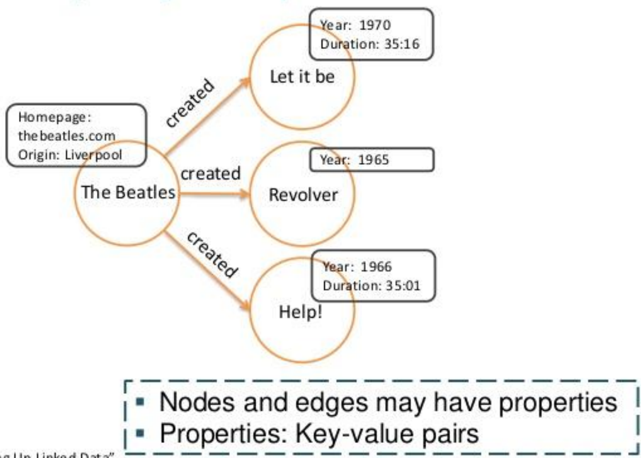




Labeled Property Graph Data Model



Property Graph Model



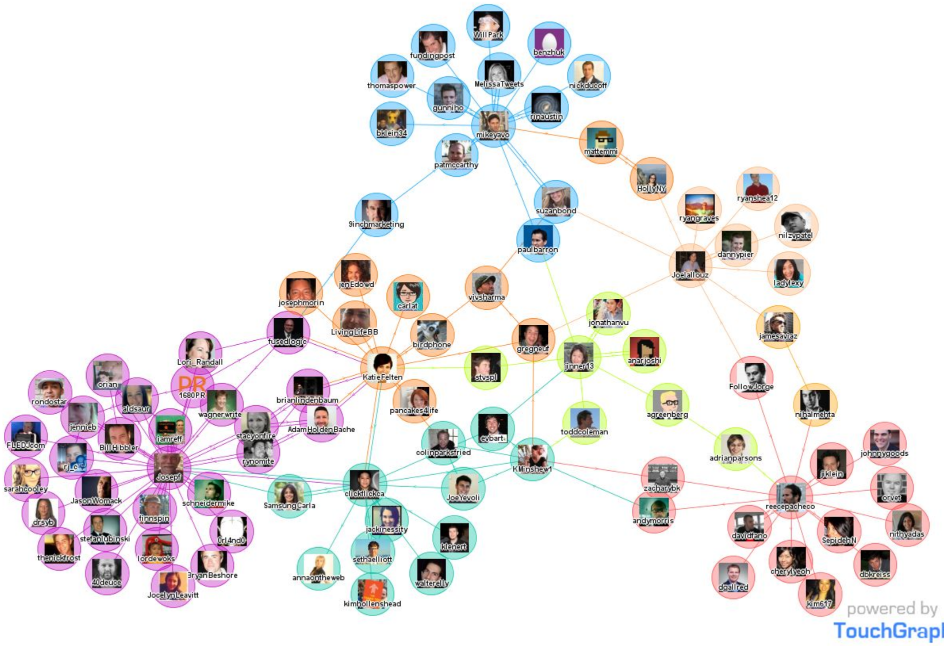
Source: "Scaling Up Linked Data". EUCLID project.

So, What is a GraphDB?

- Data model is represented by nodes and relationships
- Uses graph structures to semantically represent objects and relationships
- Relationships are first class citizens and can have properties on their own
- Allows simple and fast retrieval of complex hierarchical structures
- Directly relates data items in the store to allow data to be linked together

Typical Use Cases

- Social Networks
- Recommendations engines
- Path Finding (How do I get from x to y in the shortest path)
- Network Topology diagrams



Building blocks

- ✔ Nodes
- ✔ Relationships
- ✔ Properties
- ✔ Labels

Nodes

- Nodes represent entities and complex types
- Nodes can contain properties
- Each node can have different properties

Relationships

- Every relationship has a name and direction
- Relationships can contain properties, which can further clarify the relationship
- Must have a start and end node

Labels

- Used to represent objects in your domain (e.g. user, person, movie)
- With labels, you can group nodes
- Allows us to create indexes and constraints with groups of nodes

Graph Databases

Data Model:

- Nodes and Relationships

Examples:

- Neo4j, OrientDB, InfiniteGraph, AllegroGraph

Graph Databases: Pros and Cons

Pros:

- Powerful data model, as general as RDBMS
- Connected data locally indexed
- Easy to query

Cons

- Sharding (lots of people working on this)
  - Scales UP reasonably well
- Requires rewiring your brain

What are graphs good for?

Recommendations

Business intelligence

Social computing

Geospatial

Systems management

Web of things

Time series data

Product catalogue

Web analytics

Scientific computing (especially bioinformatics)

Indexing your *slow* RDBMS

And much more!

What is a Graph?

What is a Graph?

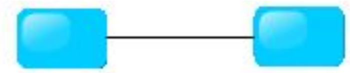
An abstract representation of a set of objects where some pairs are connected by links.

Object (Vertex, Node)

Link (Edge, Arc, Relationship)

Different Kinds of Graphs

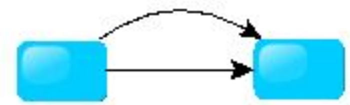
Undirected Graph



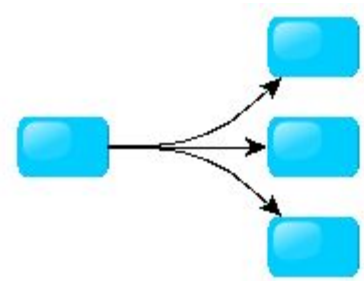
Directed Graph

Pseudo Graph

Multi Graph

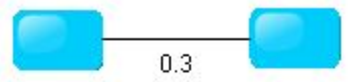


Hyper Graph

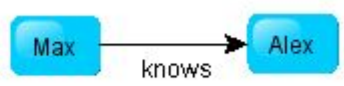


More Kinds of Graphs

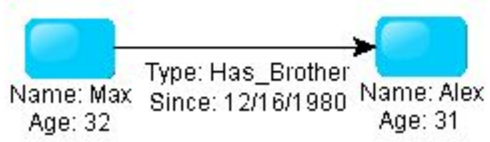
Weighted Graph



Labeled Graph



Property Graph



What is a Graph Database?

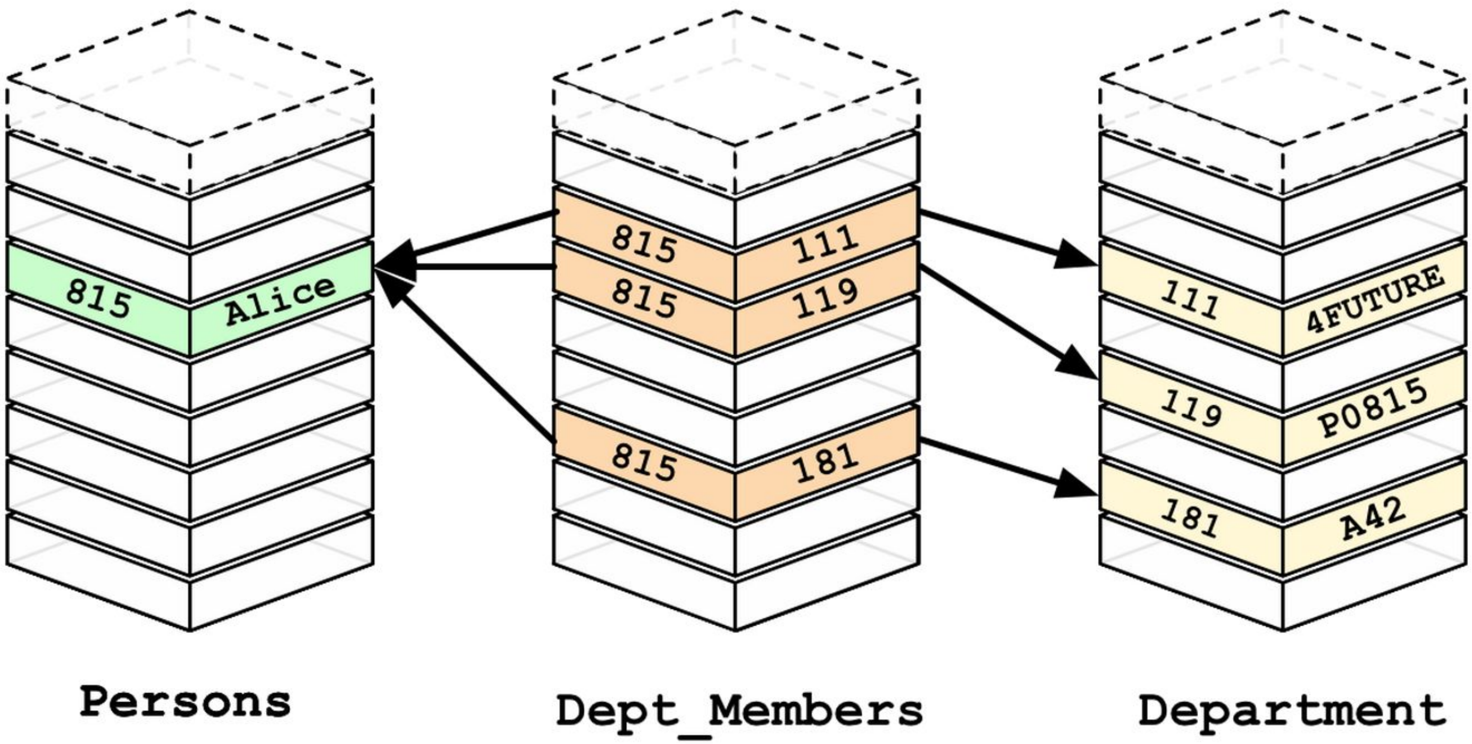
A database with an explicit graph structure

Each node knows its adjacent nodes

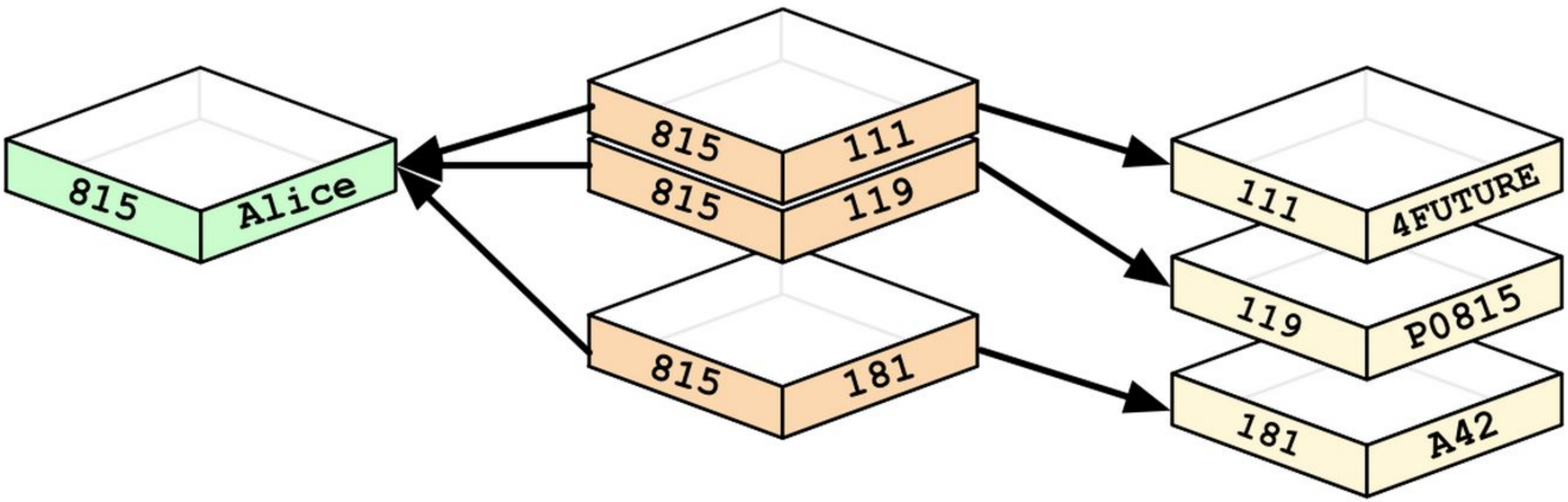
As the number of nodes increases, the cost of a local step (or hop) remains the same

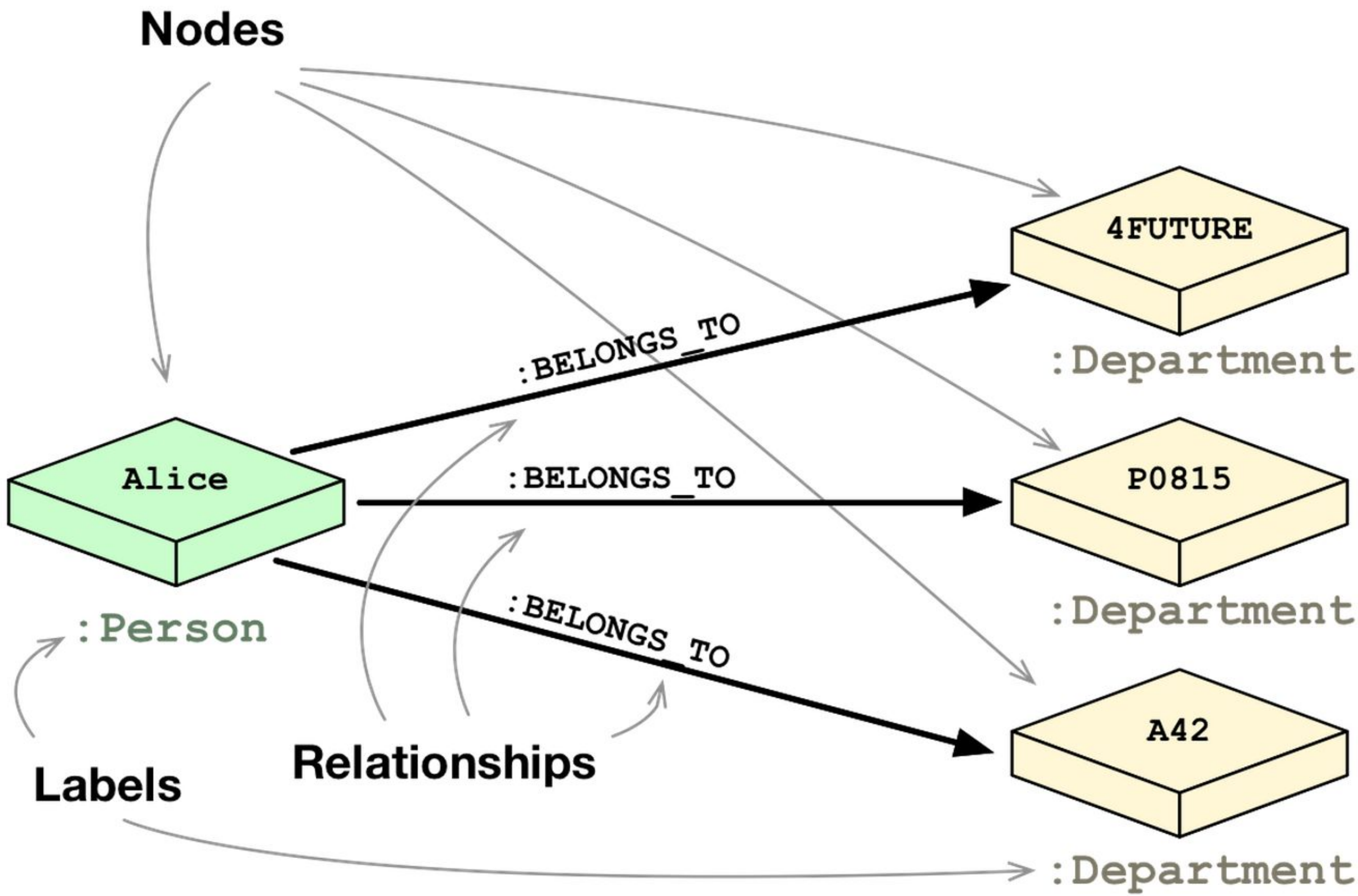
Plus an Index for lookups

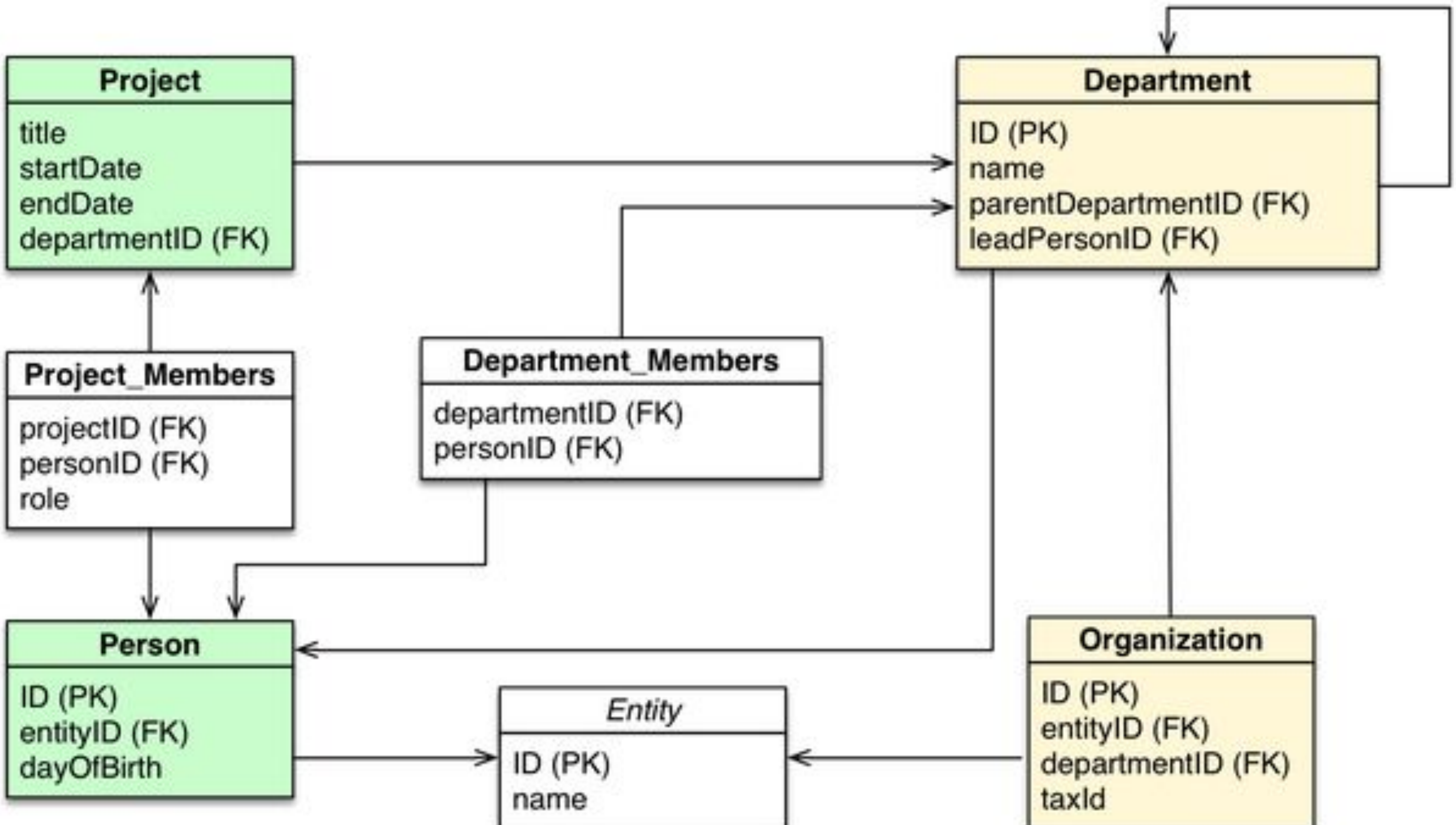
Relational Databases

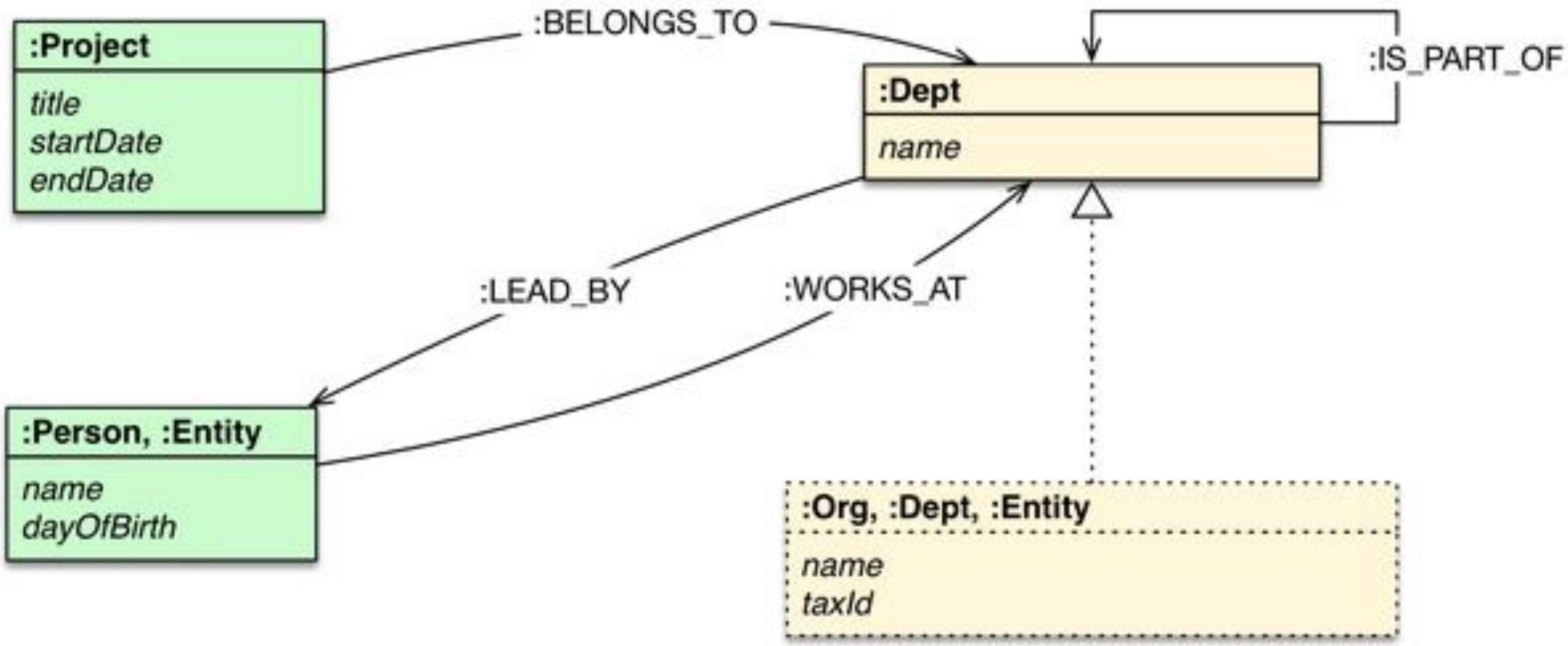


Graph Databases




Nodes
4FUTURE
:Department
:BELONGS_TO
Alice
:BELONGS_TO
P0815
:Department
:Person
:BELONGS_TO
A42
Relationships
Labels
:Department


Project
title
startDate
endDate
departmentID (FK)
Department
ID (PK)
name
parentDepartmentID (FK)
leadPersonID (FK)
Project_Members
projectID (FK)
personID (FK)
role
Department_Members
departmentID (FK)
personID (FK)
Person
ID (PK)
entityID (FK)
dayOfBirth
Entity
ID (PK)
name
Organization
ID (PK)
entityID (FK)
departmentID (FK)
taxId


:Project
title
startDate
endDate
:BELONGS_TO
:Dept
name
:IS_PART_OF
:LEAD_BY
:WORKS_AT
:Person, :Entity
name
dayOfBirth
:Org, :Dept, :Entity
name
taxId

Neo4j Tips

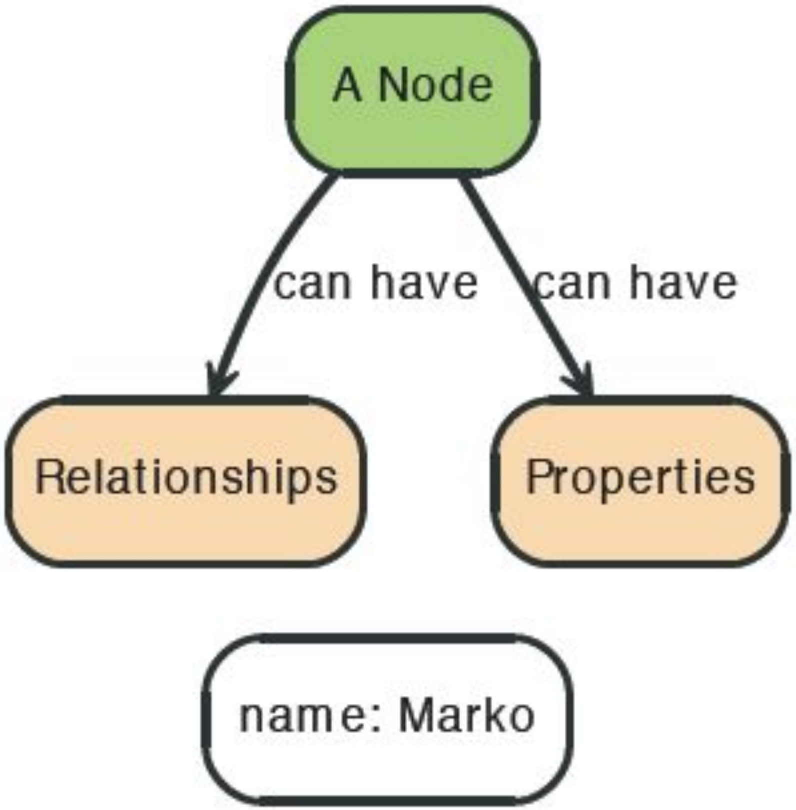
Each entity table is represented by a label on nodes

Each row in a entity table is a node

Columns on those tables become node properties.

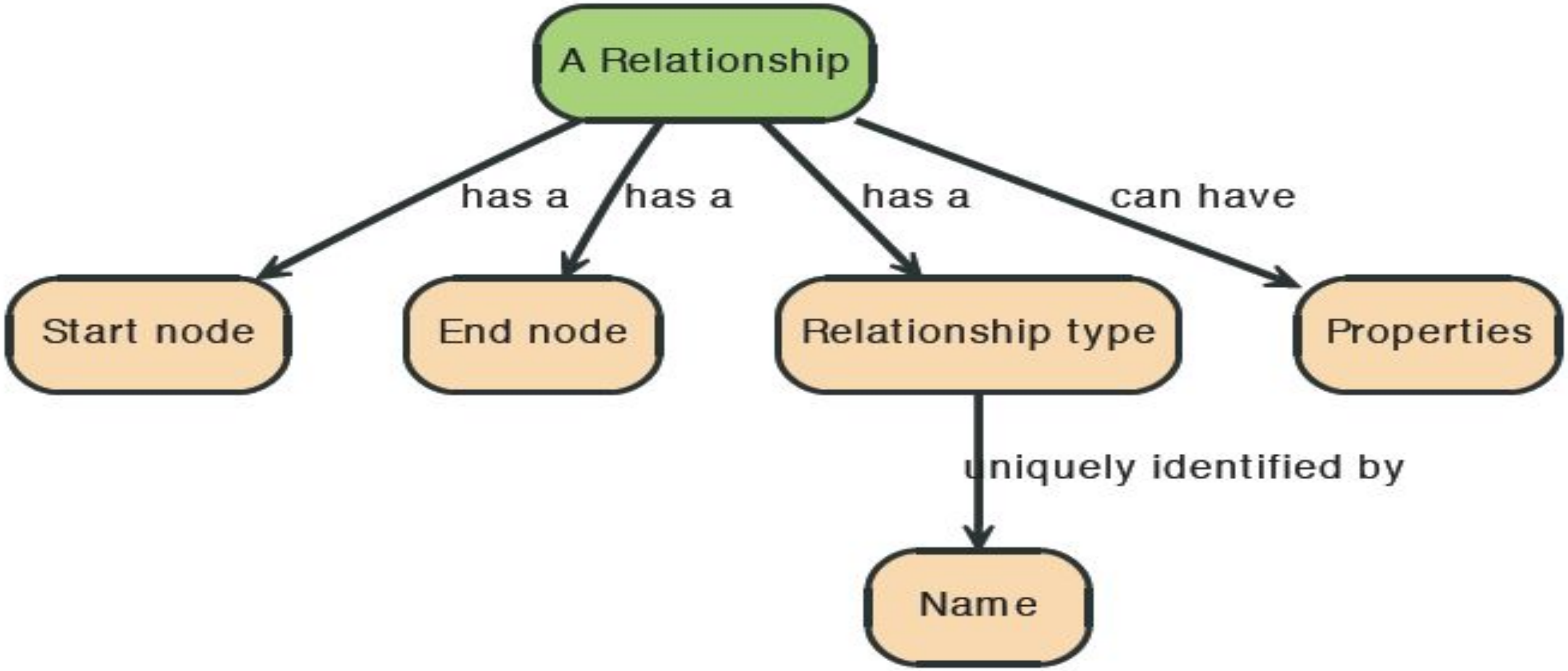
Join tables are transformed into relationships, columns on those tables become relationship properties

Node in Neo4j

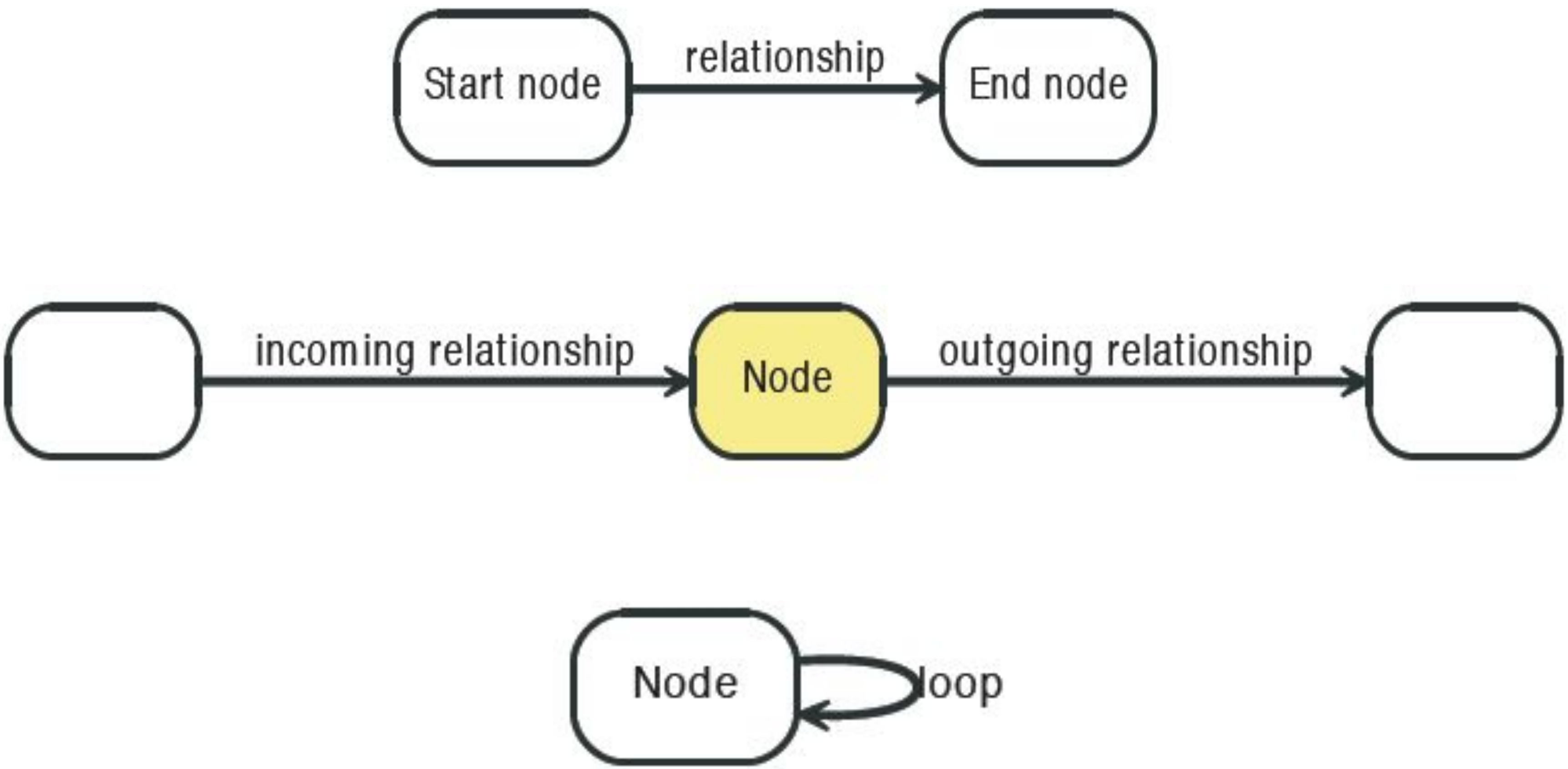


Relationships in Neo4j

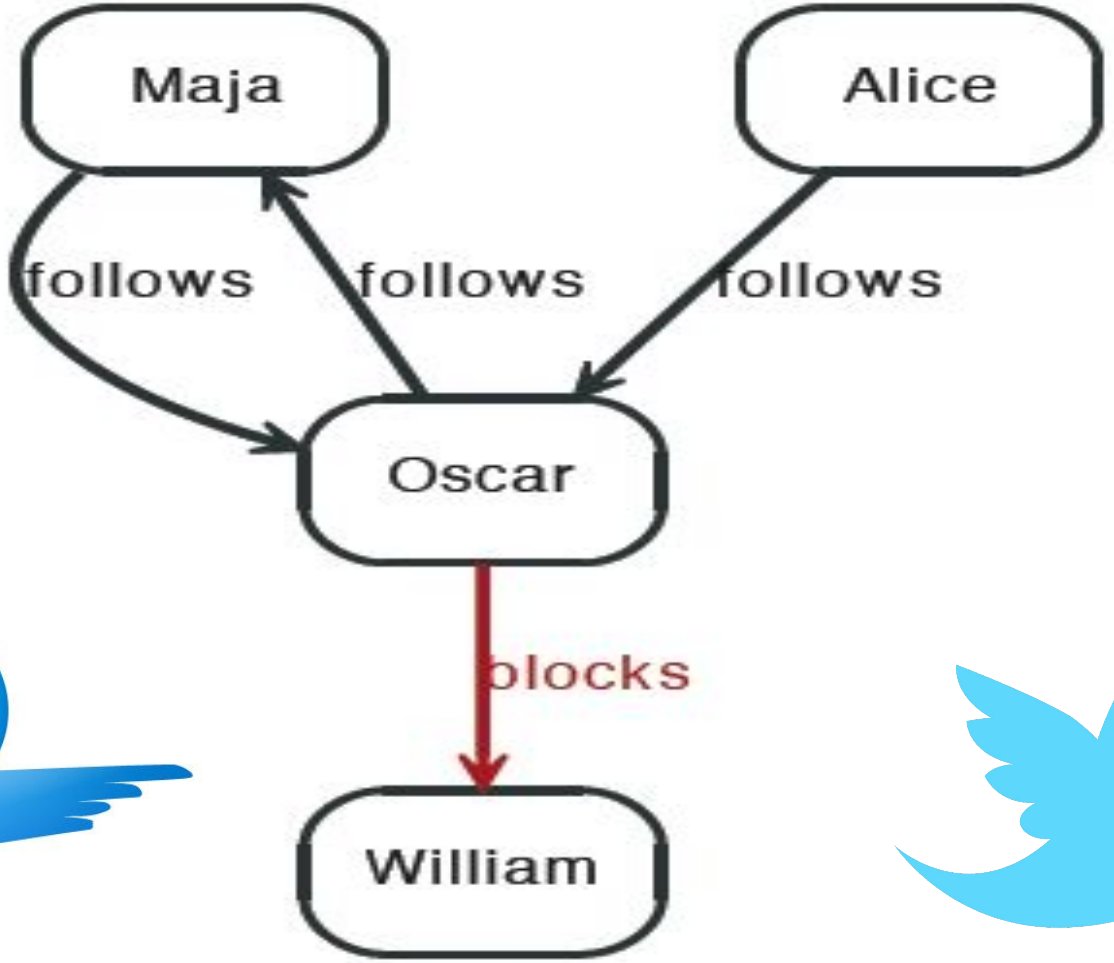
Relationships between nodes are a key part of Neo4j.



Relationships in Neo4j



Twitter and relationships



Properties

Both nodes and relationships can have properties.

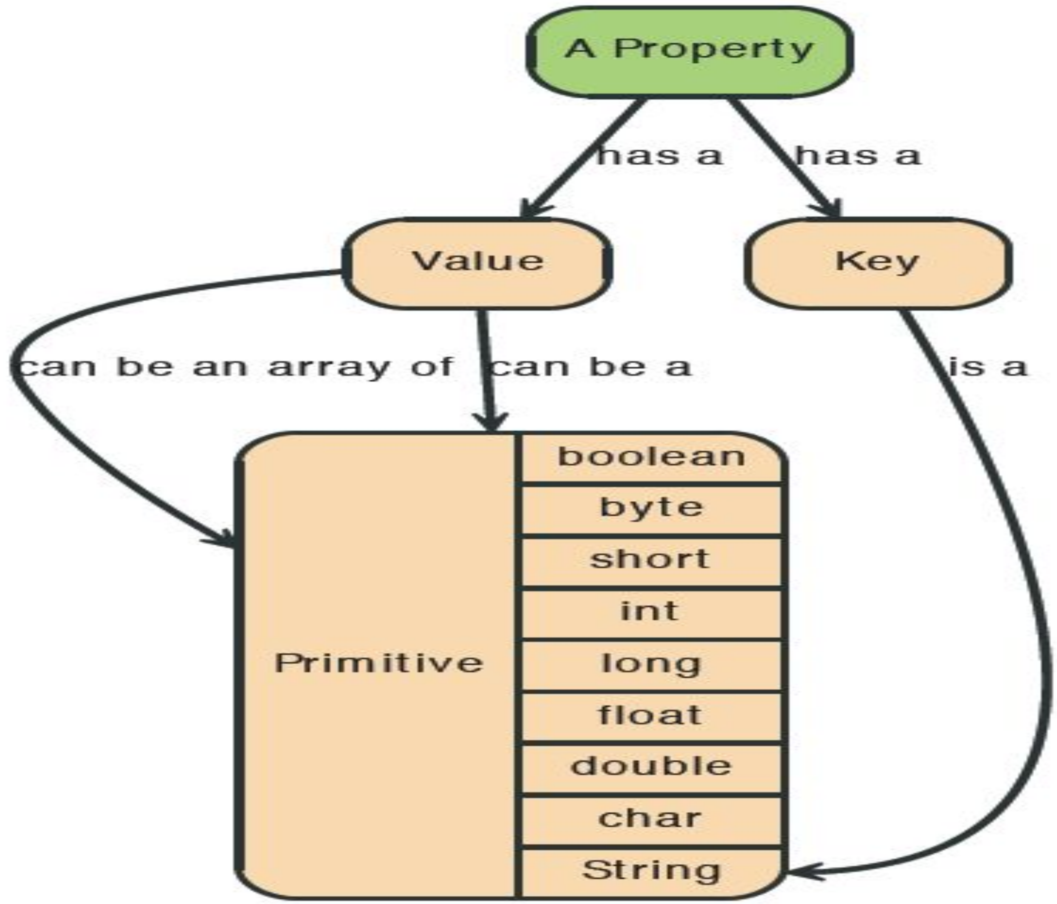
Properties are key-value pairs where the key is a string.

Property values can be either a primitive or an

array of one primitive type.

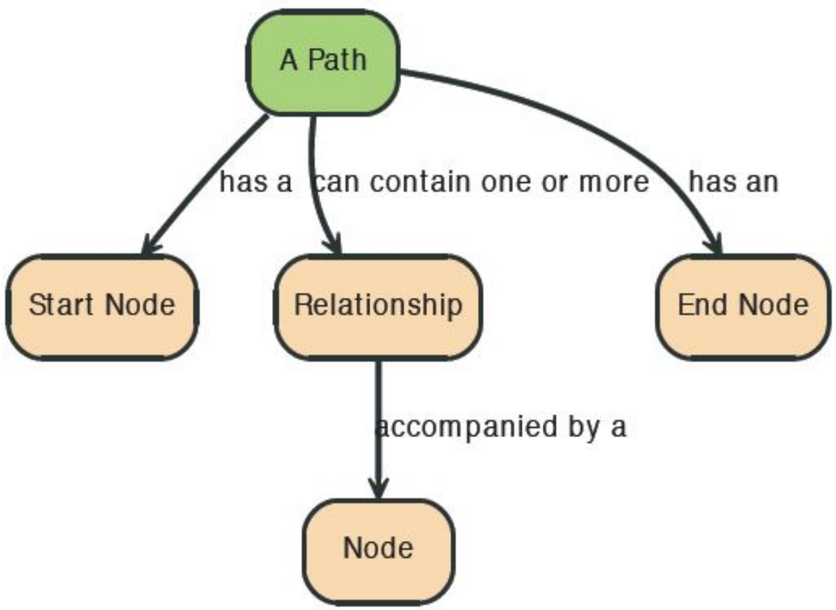
For example String, int and int[] values are valid for properties.

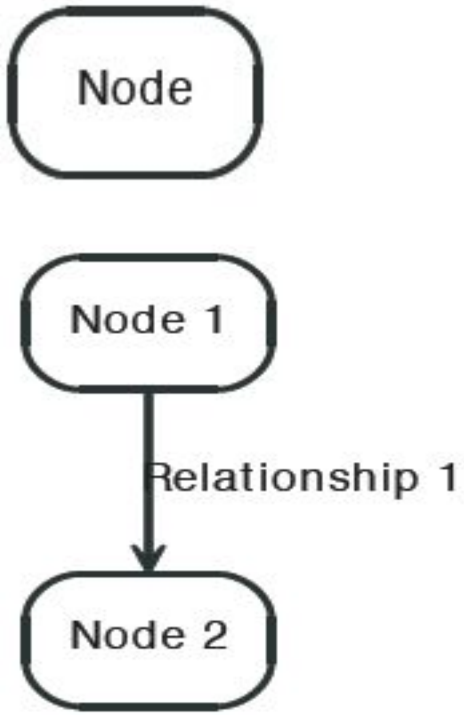
Properties


A Property
has a
has a
Value
Key
can be an array of
can be a
is a
Primitive
boolean
byte
short
int
long
float
double
char
String

Paths in Neo4j

A path is one or more nodes with connecting relationships, typically retrieved as a query or traversal result.





Starting and Stopping

```
graphDb = new EmbeddedGraphDatabase( DB_PATH );
registerShutdownHook( graphDb );
```

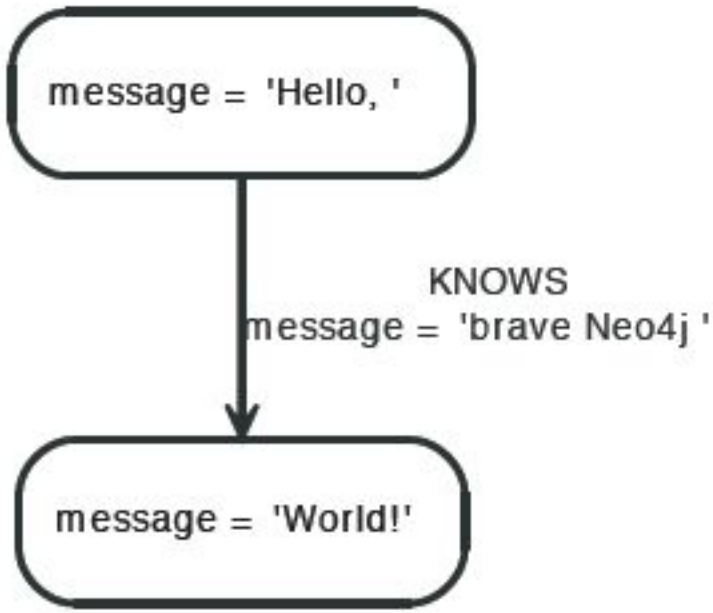
```
graphDb.shutdown();
```

```
private static void registerShutdownHook( final GraphDatabaseService graphDb )
{
    // Registers a shutdown hook for the Neo4j instance so that it
    // shuts down nicely when the VM exits (even if you "Ctrl-C" the
    // running example before it's completed)
    Runtime.getRuntime().addShutdownHook( new Thread()
    {
        @Override
        public void run()
        {
            graphDb.shutdown();
        }
    } );
}
```

Creating a small graph

```
firstNode = graphDb.createNode();
firstNode.setProperty( "message", "Hello, " );
secondNode = graphDb.createNode();
secondNode.setProperty( "message", "World!" );

relationship = firstNode.createRelationshipTo( secondNode, RelTypes.KNOWS );
relationship.setProperty( "message", "brave Neo4j " );
```



Print the data

```
System.out.print( firstNode.getProperty( "message" ) );
System.out.print( relationship.getProperty( "message" ) );
System.out.print( secondNode.getProperty( "message" ) );
```

Remove the data

```
firstNode.getSingleRelationship( RelTypes.KNOWS, Direction.OUTGOING ).delete();
firstNode.delete();
secondNode.delete();
```

The Matrix Graph Database